

“Under What Authority” as Standalone Accountability Question: Authority Context as Architectural Commitment in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 4, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the third of the four accountability questions named in the source paper’s path-retraceability vocabulary — **“under what authority”** — as a standalone architectural commitment with independent operational content, separable from the writer-identity question “by whom” with which it is most closely paired and from the questions “what was decided” and “with what rationale” with which it composes to constitute the substrate’s accountability surface.

Abstract

The CKS pattern’s path-retraceability commitment requires that substrate content support reconstruction of four accountability facts about every deployment-produced piece of substrate state: what was decided, by whom, under what authority, and with what rationale (§3.1, §11.3). A separate foundational note formalizes path retraceability and the accountability vocabulary jointly; the integrating-frame note in the present decomposition formalizes the four questions as a structured set. This note formalizes “under what authority” as having independent architectural content: every deployment-produced piece of substrate content carries authority context identifying the basis on which the write was authorized; the context is class-appropriate to the writer (orchestration-rule reference for cell-produced writes, authority position for direct human writes, rule-authorship authority for rule-authoring writes); the context links to substrate-resident authority definitions, not to opaque tokens or external authorization systems; and the context is committed atomically with the content. The note states the commitment, distinguishes it from four adjacent patterns, names the failure modes that violate it, and provides an operational test for whether a system’s authority context is CKS-coherent.

1. Why “under what authority” needs to be formalized as standalone

The path-retraceability commitment in §3.1 of the source paper, taken together with §11.3’s enumeration of authoritative state, requires that the substrate support reconstruction of four facts about every piece of deployment-produced content. The integrating-frame note in this decomposition states the four-question structure; the prior two specialization notes formalize “what was decided” and “by whom” as standalone. The present note treats “under what authority.”

A class of substrate designs identifies writers but does not architecturally record the basis on which writes were authorized. Substrates in this class take several forms: those where every human write records identity but not authority position; those where LLM-mediated writes record the LLM mediator and cell context but not the orchestration rule that authorized the specific write; those where rule changes are committed without recording the rule-authoring authority that justified the change; and those where authority is held entirely in external authorization systems and the substrate carries only references requiring external resolution. In each, “by whom” is recorded but “under what authority” is not, and the substrate cannot answer the second question from its own state. Naming the second question as a standalone architectural commitment is what makes the absence visible as a defect rather than as a tolerable simplification.

The standalone treatment is also strategically consequential. The architectural commitment to authority context being substrate-resident with linkage to substrate-resident authority definitions forecloses architectures where authority is held in external authorization infrastructure and substrate carries only authorization tokens. And it is what makes governance architectural rather than procedural in the §3.3 sense: without authority recorded as substrate content with substrate-resident linkage, authority is held in process layers — workflow systems, identity providers, authorization services — and the architectural-property qualifier fails.

2. The “under what authority” commitment, defined precisely

In the CKS pattern, the substrate satisfies the “**under what authority**” commitment when every piece of substrate content produced by deployment activity carries authority context that meets four operational components.

(a) Authority context is present for every deployment-produced write. For every decision the substrate covers under the “what was decided” commitment — cell executions, direct human overrides, rule authoring — the corresponding substrate content carries non-empty authority context identifying the basis on which the write was authorized. There is no class of deployment-produced substrate content that lacks authority context.

(b) Authority context is class-appropriate. The architecture supports three classes of authority context, corresponding to three classes of writers, and the substrate records the class-appropriate context for each write. Implementations that record the same authority context for all classes — collapsing rule-authorized cell writes and direct human writes into a single flat field — fail the commitment.

The first class is **cell-produced writes**, whether LLM-mediated or non-LLM-mediated (per the labor-allocation modes in A1.12). The authority context is the orchestration-rule reference that authorized the write per Property B from A2.20. The reference identifies the substrate-resident rule under whose authority the cell wrote what it wrote.

The second class is **direct human writes** — overrides, edits, deletions performed by humans exercising the rights formalized in A1.01 and A2.03. The authority context is the human’s authority position in the deployment’s governance structure: the substrate-resident specification of what authority that human held at write time. The reference identifies the position, not merely the human’s identity

(which is the “by whom” content). One human acting under one position at one time and another at another time is architecturally distinguishable on the authority axis only because the position is recorded.

The third class is **rule-authoring writes** — the design-time moment from A2.04 in which humans author or revise the orchestration rules that govern subsequent cell behavior. The authority context is the rule-authorship authority: the substrate-resident specification of who is authorized to author or revise rules in which scopes. Rule-authorship authority is meta-level relative to the authority of cell writes (which exist downstream of the rules) and distinct from direct-write authority (which exercises override rights without altering future cell behavior).

(c) Authority context references substrate content. Authority context is a reference to substrate content that defines the authority — to orchestration rules (substrate content per A2.04), to the authority structure (substrate content per A1.01), to rule-authorship authority (substrate content of the same kind). The reference is retraceable from substrate alone, per the substrate-only-paths property formalized in A2.41. Authority context recorded as references to external systems — OAuth scopes, SAML assertions, identity-provider claims — fails this component because resolving the reference requires consulting state outside the substrate.

(d) Authority context is committed atomically with the content. The cell→substrate write per A2.10 commits substrate content and authority context as a single architectural commit; there is no intermediate state in which substrate content exists without its authority context, and no separate post-write enrichment step adds authority context after the fact.

The four components together define the architectural commitment. A system that satisfies fewer than four cannot reliably answer “under what authority” for all deployment-produced substrate content from substrate alone.

3. What the commitment does NOT claim

The standalone treatment is not a maximalist treatment.

It does not claim authority enforcement at write time. Authority context is the metadata recorded when the write commits; authority enforcement is the runtime check that prevents unauthorized writes from succeeding. The architectural commitment is to context recording, not to enforcement mechanisms. Deployments may use any of several technical mechanisms for enforcement — capability checks, policy engines, signature verification, role-based access control — without affecting whether the architectural commitment to authority context is satisfied.

It does not specify authority-context format. Authority context can be encoded as substrate addresses, structured identifiers, foreign-key references, or any other addressable form, provided the encoding links to substrate-resident authority definitions and supports the class-appropriateness and retraceability components.

It does not require centralized authority administration. The architecture supports decentralized authority structures, delegation, layered hierarchies, and scope-restricted authority. The commitment is to the substrate carrying class-appropriate authority context, not to a particular pattern of authority

administration.

It does not require authority to be expressed in any particular language. Orchestration rules may be in natural language, structured DSL, or executable code, per A2.04; authority structure may be expressed in any form, per A1.01.

It does not specify how authority context propagates through derived writes. Where a cell write produces downstream cell executions or further writes (per substrate-mediated cell-to-cell flows from A2.12), each downstream write carries its own authority context based on the rule that authorized it. The commitment is to per-write authority context, not to context inheritance.

4. What the commitment is NOT

Four adjacent patterns are commonly conflated with the “under what authority” commitment.

Not permission-only attribution. Some implementations record write permissions — whether the writer was permitted to make the write — without recording the authority basis on which permission applied. Permission-only attribution answers “was this allowed?” but not “under what authority?” Two writers with identical permissions but different authority bases are indistinguishable under permission-only attribution, distinguishable under the present commitment.

Not role-only attribution. Some implementations record writes with role identifiers (“administrator,” “reviewer”) as authority context. Roles are abstractions over authority positions; multiple humans may share a role but hold different scopes within it, and intra-role authority may be revised over time without changing the role label. Role-only attribution loses these distinctions.

Not free-form authority claims. Some implementations record authority as free-form text supplied by the writer at write time (“acting under emergency authority”). Free-form claims are recorded but do not link to substrate-resident authority definitions; the writer’s claim is captured as text, not as a verifiable architectural reference.

Not post-hoc authority inference. Some implementations infer authority context after the fact, by analyzing the write’s content, the writer’s typical patterns, or external systems’ state at write time. Post-hoc inference reconstructs authority rather than recording it; the commitment requires authority context recorded at write time, atomically with the content.

5. Why “under what authority” is load-bearing for downstream commitments

The **human-governed commitment** from A1.01 operationalizes human authority through humans’ rights over substrate. Authority context is what makes the operationalization architecturally specific: without it, claims that humans hold authority would not be retraceable to the substrate writes those humans authorized.

The **AI-as-substrate-mediator commitment** from A1.04, with Property B from A2.20, holds that LLM mediators write under orchestration rules. The rule reference recorded as authority context is what makes LLM writes architecturally distinguishable as rule-governed rather than autonomous.

The **architectural-property qualifier** from §3.3 of the source paper, formalized further in A2.06, requires governance to be architectural rather than procedural. Authority context recorded as substrate content is what makes authority architectural; authority held only in external systems would force governance to depend on those systems' availability and policy.

The **substrate-only-paths property** from A2.41 holds that paths through substrate — including authority transitions: write authorized by rule, rule authored by human, human's authority defined by structure — must be retraceable from substrate alone. Authority context with substrate-resident linkage is what makes those transitions traceable without external resolution.

The **rule-authoring moment** from A2.04 includes rule changes whose authority context is rule-authorship authority. Naming this class explicitly makes rule evolution traceable to the humans who authored the changes under their meta-authority.

6. Failure modes that violate the commitment

A system can fail the “under what authority” commitment specifically, even when it satisfies the other accountability commitments. Nine failure modes name the most common ways this happens.

(a) External-authorization-system reliance. Authority context is held in an external identity or authorization service, and substrate carries only references that require resolution against the external system to identify the authority basis. Substrate-only paths per A2.41 are broken: a reader of the substrate cannot answer “under what authority” without consulting external state. This mode is particularly common in enterprise deployments because external authorization infrastructure (OAuth, SAML, identity-as-a-service) is operationally familiar; the trade-off is often invisible until historical decisions need to be audited and the external-authorization context has changed or expired.

(b) Permission-only attribution. Writes record whether they were permitted but not the authority basis on which the permission applied. The substrate answers “was this allowed?” but not “under what authority?”, a strictly weaker fact.

(c) Role-only attribution. Writes record role identifiers instead of specific authority positions. Multiple humans in the same role become indistinguishable on the authority axis, and intra-role authority revisions over time are not captured.

(d) Free-form authority claims. Writers supply authority claims as free text; the substrate records the claim but it does not link to substrate-resident authority definitions. Authority is recorded but not architecturally verifiable.

(e) Class-flat authority. All writes — human, LLM-mediated cell, non-LLM-mediated cell — record authority context in a single flat field, without architectural distinction between rule references (for cells) and authority positions (for humans), and without separate treatment of rule-authoring writes.

(f) Authority drift after structure changes. When the authority structure changes — a human's authority position is revised, a rule's scope narrowed — the implementation updates authority context on past writes to reflect the current structure rather than preserving the structure as it stood at write time. The substrate no longer records what authority was held when the write occurred.

(g) Implicit authority. Some writes have no authority context recorded; the system implicitly assumes the writer’s current authority is the authority basis. Reading the substrate cannot reconstruct the authority basis even in principle.

(h) Token-based authority without substrate linkage. Authority context is recorded as opaque tokens (signed credentials, capability tokens, OAuth bearers) without linkage to substrate-resident authority definitions. The token may be verifiable in some external sense but does not link to substrate; reading the substrate, one knows a token was presented but not what substrate-resident authority the token represented. The linkage required by component (c) of section 2 is broken.

(i) LLM-write authority recorded as LLM identity. LLM-mediated writes record the LLM mediator as the authority context, rather than recording the orchestration rule that authorized the write. The architectural distinction between LLM identity (which is “by whom” content per A2.37) and rule reference (which is “under what authority” content) collapses, and the substrate cannot answer whether the LLM wrote under one rule or another for any given write.

A system exhibiting any of (a)–(i) does not satisfy the commitment.

7. Operational test

A system satisfies the “under what authority” commitment if and only if all of the following are true at all times during the substrate’s existence.

1. Every piece of substrate content produced by deployment activity carries non-empty authority context.
2. Authority context is class-appropriate: cell-produced writes record an orchestration-rule reference; direct human writes record the human’s authority position; rule-authoring writes record the rule-authorship authority. The classes are architecturally distinguishable in the substrate’s representation.
3. Authority context links to substrate content that defines the authority — orchestration rules per A2.04, authority structure per A1.01, rule-authorship authority of the same kind — with the linkage retraceable from substrate alone per A2.41.
4. Authority context is committed atomically with the content at the cell→substrate write per A2.10. There is no readable substrate state in which content exists without its authority context.
5. Authority context is preserved across authority-structure evolution; the recorded authority reflects the structure as it stood at write time, not the present structure.

A system that fails any of (1)–(5) does not satisfy the commitment in the architectural sense, even if it provides authority tracking in some other form. Such a system is not CKS-coherent on the authority axis, and downstream work that relies on its authority guarantees should be scoped accordingly.

8. Why naming “under what authority” as standalone matters

Implementations under pressure to integrate with enterprise authorization infrastructure, simplify governance overhead, or unify authority representation across writer classes drift consistently toward authority patterns that fail one or more of the architectural components. The drift is steady because external authorization infrastructure is operationally familiar, role-based abstractions are convenient, token-based credentials are widely supported, and class-flat authority fields are simpler to schema. The result is systems where authority is held outside substrate, where LLM-mediated writes are not architecturally distinguishable as rule-governed, where authority-structure changes silently update past attribution, and where substrate-only paths are broken because authority resolution requires external systems.

Naming “under what authority” as a standalone architectural commitment — with the four components specified in section 2, the limits in section 3, the four adjacent-pattern distinctions in section 4, the load-bearing connections in section 5, and the nine failure modes in section 6 — gives downstream implementers a precise specification of what the architectural commitment to authority context requires, independent of how the other accountability questions are answered. The companion specialization notes formalize “what was decided” and “by whom” earlier in the decomposition and “with what rationale” subsequently; the note on the six provenance metadata fields specifies the per-write architectural surface that supports all four questions, and the note on substrate-only paths specifies the retraceability property authority context’s linkage component depends on.

Subsequent work that implements, extends, or argues against the CKS authority commitment should use “under what authority” in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). “Under What Authority” as Standalone Accountability Question: Authority Context as Architectural Commitment in the Coordination Knowledge Substrate Pattern. May 4, 2026. ORCID: 0009-0004-8065-3235.